# HW2 : AI-Assisted Programming Project

## SeatView 3D 座位視野模擬器

Nov 1, 2026 · 112502546 張宇頡

本專案 SeatView 是一個開源的 3D 座位視野模擬器：輸入「區、排、號」，就能看到從該座位望向舞台（演唱會）或本壘板（棒球）的畫面，並顯示距離、俯角與水平偏角。從 10/23 到 11/1 共約 180 筆 commit，已做出台北大巨蛋棒球／演唱會兩種配置，並提供 MCP server 讓 AI agent 直接查詢座位視野。

**線上試用**：https://benevolent-sfogliatella-3ed179.netlify.app。指定座位的範例網址：棒球 113 區 38 排 7 號、演唱會 201 區 15 排 15 號。本機執行、測試與 MCP 接法見 repo 的 README。

## 1. 專案目的

**要解決的痛點**：每次買演唱會或球賽門票時，票務系統只給座位圖，看不出實際視野。同一個票價的兩個座位，可能一個正對舞台、一個被斜角或仰角吃掉一半畫面，買之前沒有數字可以比。

**我的做法**：

- 場館與座位由 JSON 描述，座位的位置、相機、距離、仰角都由純函式計算，three.js 只負責畫。
- 同一座場館可以有多個 layout(版面配置)，例如演唱會、棒球比賽，每個 layout 各自有焦點，如舞台中心、本壘板與 overlays(疊加物件)。
- 核心計算不依賴 three.js 與 DOM，所以網頁、unit test、MCP server 三者共用同一套算式，數字不會不一致。

**與 twconcertview.com 的關係**：它是我的靈感來源，但它不是開源專案，因此我沒有複製它的程式碼、3D 模型或圖片。所有場館資料與模型都是自己建立：

- 示範場館 `demo-arena` 完全是自編的資料。
- 台北大巨蛋是「近似模型」：外框來自 OpenStreetMap（© OpenStreetMap contributors，ODbL），repo 只放擬合後的超橢圓，不放原始座標；票區編號來自公開座位圖與我自己整理的票面標籤。
- 本機的 `seatpicture/` 參考照片資料夾已加入 `.gitignore`，照片不進 repo；文件只記錄從照片取得的數字，截圖只用本專案自己的 render 畫面。
- 第三方程式庫只用寬鬆授權：three.js（MIT）、官方 MCP TypeScript SDK（MIT）、zod、Vite、Vitest。

**為什麼先做這個範圍**：作業時間有限，我先把可驗證的範圍做扎實（純函式加測試），再往上疊畫面。這也是為什麼第二階段（棒球版）能提早在大巨蛋上做出來：架構一開始就留好 layout 擴充點。

### 開發歷程

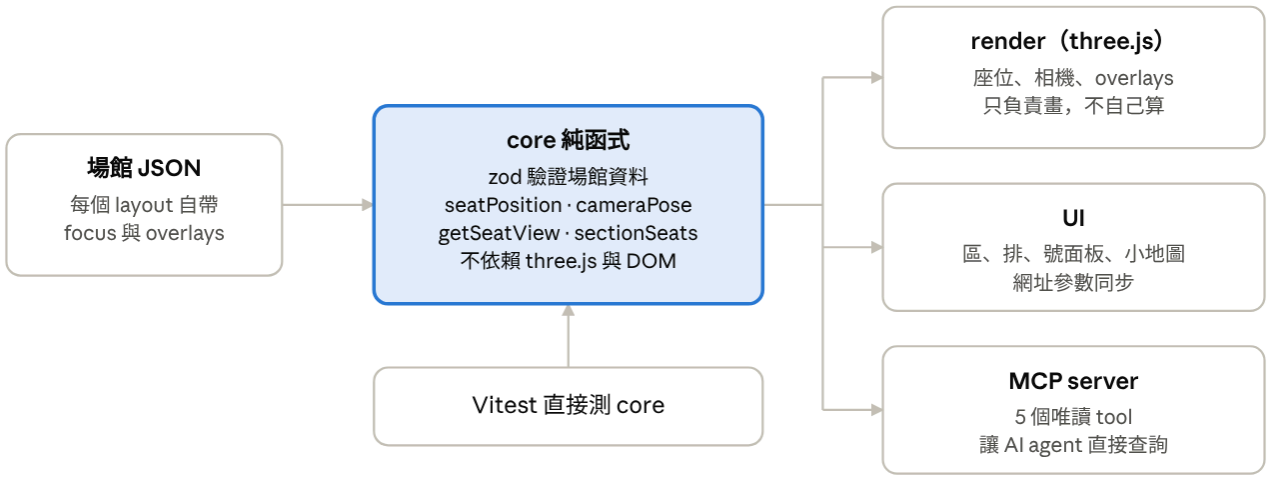
從 10/23 的雛型，到 11/01 的現在3D模型，六個階段的 render 畫面如下。每一階段都對應 git log 中的一組 commit（測試再實作）。




前三張是 10/23 一天內完成：先把 core 算對，再把真實場館放進來驗證「一個場館多個 layout」的架構。後三張是把資料換成官方票區，再慢慢追加介面與擬真度。這個順序也是我刻意安排的，因為畫面可以不斷優化，但核心的數值必須先測試正確通過。

## 2. 系統設計與架構

架構的主軸是「資料 → 純函式 → 畫面」單向分層，three.js 只出現在最外層，所以最重要的計算都能在 Node 裡直接測試。




架構分層 · 資料到 core 再到三個使用端

箭頭代表資料流向：場館 JSON 經 core 驗證與計算後，才交給畫面、面板與 MCP，最外層三個使用端都不重寫算式。

| 層 | 位置 | 負責 | 刻意不做 |
|---|---|---|---|
| 場館資料 | `src/venues/*.json` | 場館、區、layout（各自的 focus 與 overlays） | 不逐席列座標 |
| core | `src/core/` | zod schema 驗證、`seatPosition`、`cameraPose`、`distance`、`elevationAngle`、`horizontalAngle`、`getSeatView`、`sectionSeats` | 不 import three.js，不碰 DOM |
| render | `src/render/` | 把 core 的結果轉成 three.js 物件（InstancedMesh 座位、相機、overlays） | 不讀 section 的幾何欄位，不自己算位置與距離 |
| UI | `src/ui/` | 區、排、號選位面板、小地圖、網址參數同步 | 不含計算邏輯 |
| MCP | `src/mcp/` | 5 個 tool，輸入輸出用 zod schema | 只用 core，不重寫算式 |

## 重要設計決策：為什麼這樣選

1. **用參數描述一個區，不逐席列座標**：區有 `arc`（圓心、第 1 排半徑、起訖角度）和 `linear`（第 1 排前緣起訖點加上 `facing`）兩種。理由：手寫 JSON 後能從平面圖直接量數字，也不用維護上萬筆座標。代價：形狀不規則的區要拆成多區。

2. **座標採用 Y 軸向上，平面圖 (x, z) 跟 SVG 同向：**這和 three.js 一致，從自己畫的平面圖量到的座標可以直接填，不會左右鏡像。左右鏡像這種最容易出錯的 bug，我額外寫了專門的測試把關（見第 4 節）。
3. **區是否開放用 `section.layouts?` 白名單表示：**看台是建築，所有 layout 共用；搖滾區是活動配置，只標 `["concert"]`。新增棒球版只要加 layout，看台不用改。
4. **`Focus` 加 `facingDeg`：**只有一個點算不出「正對還是側面」，水平偏角需要它。
5. **用 zod 寫 schema：**一份定義同時得到執行期驗證與 TypeScript 型別，能抓到手打 JSON 的錯（例如 `seatsPerRow` 長度不符、`facing` 落在前緣線上）；MCP SDK 本來就需要 zod，不算多加依賴。
6. **render 只畫不算：**為了這條界線，core 新增 `sectionSeats(venue, sectionId)`，回傳每個座位的位置與朝向；render 對座位的幾何一律來自 core，網頁與 MCP 的數字才不會分岔。

### 第二階段的擴充點是否照著原算式疊加

有。同一座台北大巨蛋在同一份 JSON 裡已經同時有 `concert` 和 `baseball` 兩個 layout，棒球版的 focus 是本壘板。之後的擴充（venue meta、區的階梯結構、polygon overlay、穹頂外殼與螢幕）在 commit 上都是「先加 schema 的選填欄位，再加測試與實作」，沒有因為棒球而回頭改 `seatPosition`、`cameraPose` 這類基礎函式的用法。

## 3. 如何與 AI 協作

我把 AI 分成兩個角色，自己保留所有取捨的決定權：Claude Project 當整個專案的流程及設計助理，Claude Code 負責當實作的角色，每個重要決策都寫進 `docs/ai-log.md`，格式是「日期｜問題｜AI 建議｜我的取捨」。

| 角色 | 負責 | 約束它的東西 |
|---|---|---|
| 我 | 決定範圍、驗收、每個設計點的取捨 | — |
| Claude Project（SeatView） | 規劃、架構討論、設計決策、整理報告與 ai-log | 專案指令（作業要求、成功標準、回覆規則） |
| Claude Code | 寫測試、實作、commit | `CLAUDE.md`（架構原則、測試先行、commit 規則） |

### 協作流程

1. **先討論再實作：**例如 schema 與 core 函式簽名，我先在 Project 裡跟 AI 比較方案，確認後才進 Claude Code。

2. **把決定寫進 `CLAUDE.md` 當規則：**例如討論 render 時決定的「render 不讀 section 的幾何欄位，座標與朝向一律來自 core」，就補進 `CLAUDE.md`，之後 Claude Code 每一輪都會讀到。
3. **測試先行：**Claude Code 先 commit 一個會失敗的測試（commit 訊息標 `(red)`），再 commit 讓它變綠的實作。
4. **每個小功能一個 commit**：訊息用 `feat(core):`、`test(render):` 這種前綴。
5. **每輪結束追加 ai-log：**記錄 AI 建議了什麼、我選了什麼、為什麼。

## 關鍵決策（節錄自 ai-log）

| 日期 | 議題 | AI 建議 | 我的取捨 |
|---|---|---|---|
| 10/23 | 場館 JSON 與 core 簽名 | 座標慣例、`arc`／`linear` 參數化、`layouts?` 白名單、`Focus.facingDeg`、zod、新增 `getSeatView` | 全部同意；要求 `demo-arena` 補一個最小的 baseball layout，否則 `SECTION_NOT_IN_LAYOUT` 的測試測不到；先用 demo-arena 做完 core，再做真實場館 |
| 10/23 | render 設計 | 五個確認點：新增 `sectionSeats`、座位用 InstancedMesh、FOV 50° 等 | 五點都同意，另外加上「窄螢幕改用水平 FOV」避免直式手機把舞台左右切掉，並補測試 |
| 10/23 | 大巨蛋看台放不進 OSM 外框（30.8% 的座位在外面） | 比較四個方案，建議方案 B（本壘後方弧、兩側直線、外野同心弧），並允許二層懸挑 | 採用方案 B，`taipei-dome` 改用新的產生器重建 |
| 10/23 | MCP server | 放 `src/mcp`、Node 直接跑 TypeScript、5 個 tool、不加 `@types/node` | 全部採用 |
| 11/01 | B1 第 1 排降到場地高度 | 掃描後發現「左右對稱」加「離界外線 ≥ 7 m」的條件下所有組合都放不下；唯一的解是不對稱，AI 依規則停下回報 | 我決定不採用不對稱解；視野文章裡放不下的 15 個座位標籤改成「只回報」，不為了湊數改模型 |
| 11/01 | 依照片校色 | 指定的照片路徑不存在，AI 停下來問；之後 3 輪調整 | 同意換取樣框；座椅最亮群差 −22 時，我選擇接受現況，不放寬亮度上限 |

### AI 出過的錯，以及怎麼抓到的

- **誤提交 `.history/`**：Claude Code 在第一批測試的 commit 裡帶進 VS Code 的 Local History 資料夾。AI 自己發現並回報；因為還沒 push，直接修正那筆 commit，加入 `.gitignore`，之後 `git add` 一律指定路徑。我的觀察：如果我沒看 git log，也可能漏掉。
- **Vitest 通過、Node 卻跑不起來**：MCP 的 `ToolFailure` 用了建構子參數屬性，Vitest 能編譯、Node 的型別剝除不支援。修正後在 tsconfig 開 `erasableSyntaxOnly`，之後 typecheck 就會擋下，並實際放回語法確認會報 TS1294。
- **分析腳本的角度符號錯誤**：把左外野桿角度寫成 `180 - aR`，外野弧繞到本壘那一側，算出「外野放不下」的結論不可信。改成 `-180 - aR` 後重算，並加回歸測試。
- **測試與資料不一致**：測試要求 `SECTION_NOT_IN_LAYOUT`，但示範資料沒有 baseball layout。這是我審查測試案例時發現的，要求補上。

### 我學到的

把限制寫成測試與 `CLAUDE.md`，比口頭叮嚀可靠。遇到限制互相衝突時（例如 B1 的對稱規則），AI 是停下來回報，而不是悄悄放寬規則，讓我可以自己決定要不要改規則。AI 的自述也需要複查：commit 內容、測試是否真的先紅後綠，我都回頭看 git log 與測試輸出確認。

## 4. 軟體工程實踐

我把軟體工程當成**讓 AI 寫的程式碼可以被檢驗**的工具。下表每一項都列出做法、原因，和可以怎麼自己驗證。

| 實踐 | 做法 | 為什麼 | 怎麼驗證 |
|---|---|---|---|
| 測試先行 | 先 commit 失敗的測試（訊息標 `(red)`），再 commit 實作。紅燈階段先放「有型別的空殼」（`throw new Error("not implemented")`） | 如果測試直接 import 不存在的模組，整個檔案會一起掛，看不出哪些案例還沒做；空殼讓每個測試各自失敗 | `git log` 可看到 `test:` 與 `feat:` 成對出現（約 64 筆 test commit）。core 第一輪的 64 個失敗，用 Vitest JSON reporter 依第一行錯誤訊息分組，確認全部來自空殼 |
| 避免「假通過」 | 錯誤路徑的測試斷言丟出的是 `SeatViewError` 且 `code` 正確；schema 測試只接受 `ZodError` | 空殼丟一般 `Error` 時，這些測試不會意外變綠 | 見 `docs/ai-log.md` 的「core 實作（測試先行）」 |

| 實踐 | 做法 | 為什麼 | 怎麼驗證 |
|---|---|---|---|
| 純函式核心 | `src/core/` 沒有 three.js 或 DOM 依賴，網頁與 MCP 共用 | 能在 Node 裡直接測，不用瀏覽器 | `npm test`（Vitest）；MCP 測試確認 tool 回傳的數字與 core 的 `getSeatView` 完全一致 |
| 型別與資料驗證 | TypeScript 型別檢查、zod 驗證場館 JSON、tsconfig 開 `erasableSyntaxOnly` | 手打 JSON 容易出錯；Node 的型別剝除不支援某些語法，Vitest 卻能跑 | `npm run typecheck`；放回參數屬性會出現 TS1294 |
| 不變量測試 | 大巨蛋資料有 68 項不變量（座位離外框 ≥ 3 m、同區不重疊、排數上限、座位編號方向等） | 資料是產生器算出來的，人眼很難看出「其中一區座位穿出外框」 | `src/venues/taipei-dome.invariants.test.ts`；**突變確認**：暫時把內野六區座號方向全部反過來，六條慣例測試全部失敗，還原後全部通過 |
| 回歸測試 | 左右鏡像、角度符號錯誤都補上測試 | 這些錯誤「看起來很合理」，最容易再犯 | `src/render/mirror.test.ts` |
| 小而清楚的 commit | 約 180 筆 commit，用 Conventional Commits 前綴：`feat` 67、`test` 64、`docs` 35、`chore` 5、`fix` 4、`refactor` 3、`perf` 2 | 每個 commit 只做一件事，回頭查 bug 時能定位 | `git log --oneline` |
| 功能分支 | `official-sections`、`dome-interior`、`ui-theme`、`curved-stands`、`wide-bowl`、`three-centred` 等分支再合併 | 大型改動不影響 main | `git log --graph` |
| 人工目視清單 | `docs/manual-checks.md`：網址參數、預期、結果、截圖 | 比例感、光影、手感只能用眼睛確認 | 清單中有發現並修正的問題（鳥瞰取景、dev server 快取），截圖在 `docs/screenshots/` |

**誠實交代**：少數測試是實作後補寫的（例如 overlay 幾何、scene dispose），commit 訊息明寫 `(written after the implementation)`，沒有假裝成測試先行。另外幾筆 commit 有用 amend 整理，所以 `git reflog` 比 `git log` 多幾行。

### 資料檢驗與授權

- **數字等級**：大巨蛋的每個數字都標 A（規則或官方定義）、B（新聞報導，附來源）、C（我自己估的，只能寫成「估算」），記錄在 `docs/venue-taipei-dome.md`，MCP 的 `get_venue_provenance` 也能查。
- **免責聲明**：近似模型的查詢結果一定附免責聲明，畫面上也標「示意」，不能作為購票依據。
- **授權**：不複製 twconcertview.com 的任何資產；使用 OSM 資料時標註 ODbL 來源；參考照片不進 repo；程式庫均為寬鬆授權。

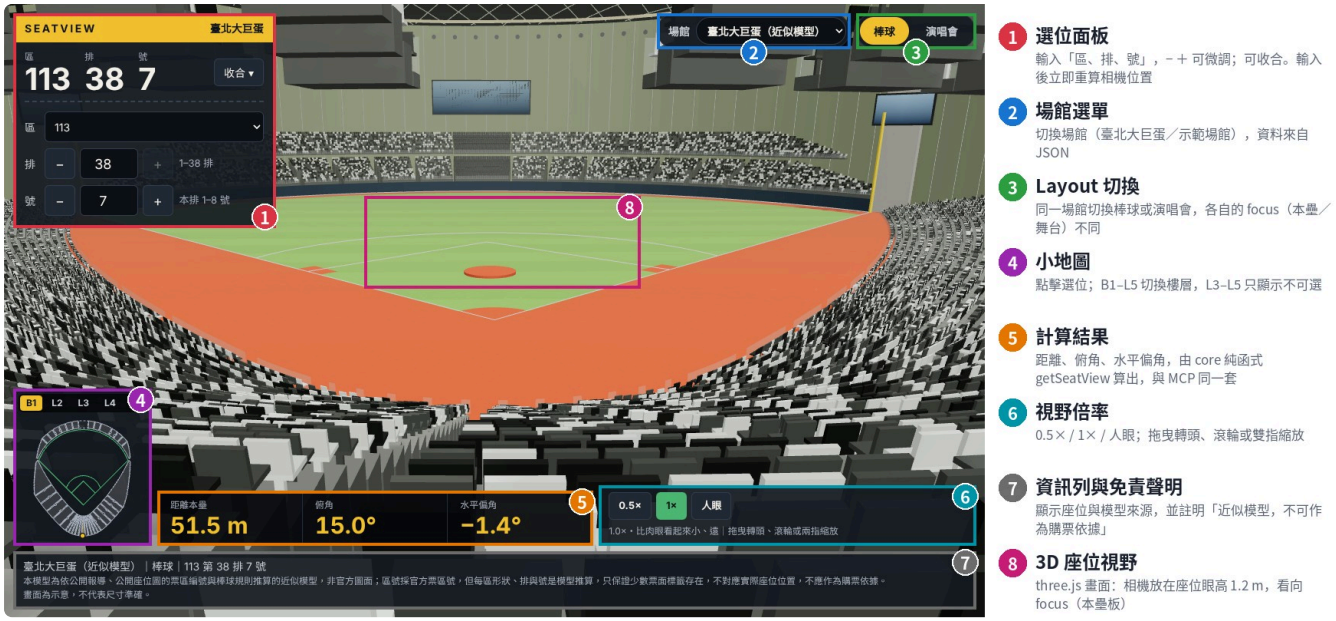
## 5. 成果與驗證

四項成功標準中，3D demo 與 unit test 已達成，MCP server 已完成且有自動化測試，報告與 ai-log 即本文與 `docs/ai-log.md`。

| 成功標準 | 結果 | 怎麼驗證的 |
| --- | --- | --- |
| 可運作的 3D 視野 demo | `npm run dev` 後用網址參數選座位，例如 `?venue=taipei-dome&layout=baseball&section=113&row=38&seat=7`；有區、排、號選位面板、小地圖、演唱會與棒球 layout 切換，資訊列顯示距離、仰角、水平偏角 | `docs/manual-checks.md` 的目視清單（預期、結果、截圖）；手機 375×812 上舞台沒被左右切掉 |
| 核心運算有 unit test 且通過 | 最後一次在 ai-log 記錄的結果：**519 個測試通過、1 個 todo**，typecheck 與 build 通過（記錄於 11/01 校色那一輪） | `npm test`（Vitest）、`npm run typecheck`；1 個 todo 是外野座號方向，因為沒有照片證據所以刻意不寫斷言 |
| MCP server 可被 agent 呼叫 | 5 個 tool，見第 6 節 | InMemoryTransport 端到端測試、真正的 stdio 連線 |
| 報告與 AI collaboration log | 本文；`docs/ai-log.md` 約 860 行，每輪一篇 | 對照 git log 的日期與 commit |

### 系統介紹：介面導覽

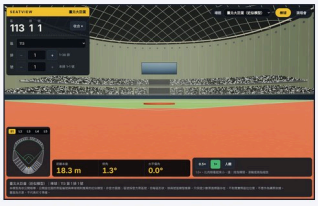
下圖是現在的網頁（113 區 38 排 7 號，棒球 layout），用編號標出每個區域的功能。其中 5 號的座位數值和 MCP 的 `get_seat_view` 來自同一個函式。




## 不同座位、不同角度的視野

同一個場館、不同座位望向本壘板。距離、俯角、水平偏角就是這些畫面的「數字版」：例如 113 區第 1 排（距離 18.3 m、俯角 1.3°）貼近本壘、視線幾乎平行，而 212 區二層（俯角 25.8°）從高處往下看。越高、越遠，俯角和距離就越大。

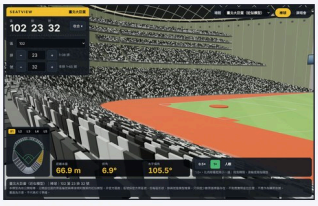
**棒球 layout：不同座位望向本壘板**



113 區 1 排 1 號（本壘正後方第一排）
距離 18.3 m 俯角 1.3° 偏角 0.0°



113 區 38 排 7 號（本壘正後方後排）
距離 51.5 m 俯角 15.0° 偏角 −1.4°



102 區 23 排 32 號（側面）
距離 66.9 m 俯角 6.9° 偏角 105.5°



122 區 20 排 13 號（另一側）
距離 52.4 m 俯角 7.7° 偏角 −96.0°



201 區 15 排 15 號（二層）
距離 67.8 m 俯角 19.7° 偏角 91.6°



212 區 10 排 3 號（二層本壘後方）
距離 46.8 m 俯角 25.8° 偏角 6.7°



146 區 14 排 1 號（外野最後一排）
距離 124.4 m 俯角 3.7° 偏角 144.9°



140 區 14 排 9 號（中外野正後方）
距離 135.9 m 俯角 3.4° 偏角 174.6°

外野座位（146、140 區）距離本壘 124–136 m，俯角只有 3.4–3.7°，因為目前模型只建了外野一層（B1、14 排），外野二層（233–244 區）還沒建模，所以目前看不到「外野高處望向本壘」的座位視野。這是已知限制，也是下一步要補的資料。

同一個場館切成演唱會 layout 後，focus 變成舞台，搖滾區 F1 在舞台正前方，最後方的 115 區 38 排距離舞台達 159.9 m。鳥瞰是除錯用的隱藏模式，用來檢查看台與場地的關係，不是正式功能。

## 演唱會 layout 與鳥瞰角度



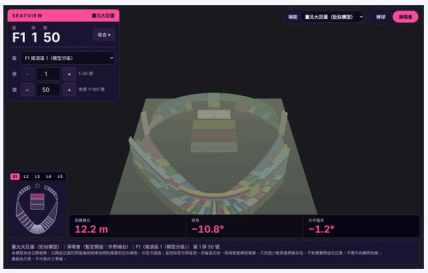

**F1 區 1 排 50 號（搖滾區前排）**
距離 12.2 m 俯角 −10.8° 偏角 −1.2°



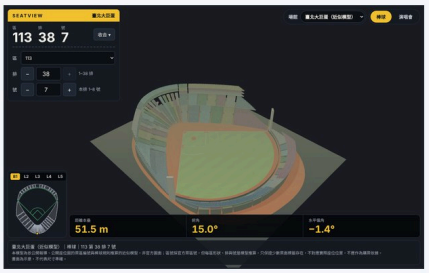

**201 區 15 排 15 號（二層）**
距離 128.1 m 俯角 9.1° 偏角 30.3°




**115 區 38 排 9 號（後方）**
距離 159.9 m 俯角 3.8° 偏角 −4.7°




**鳥瞰（除錯模式）：演唱會配置**
舞台在外野端，搖滾區 F1–F3 在舞台前




**鳥瞰：棒球，向左旋轉**
拖曳旋轉，檢查看台與場地關係




**鳥瞰：棒球，低角度**
從低角度檢查看台高低與穹頂




截圖說明：這些截圖由 Claude Code 在 `three-centred` 分支最新 commit（e5025e7）上啟動 `vite` dev server，再以 Playwright 無頭 Chromium（軟體 WebGL）截取；座位資料與分區都是模型推算，只保證少數票面標籤（如 113/38/7、102/23/32、201/15/15）存在。視覺上與實際瀏覽器的 GPU 渲染可能略有差異。

## 三個驗證案例：結果為什麼是這樣及如何確認的

**案例 1：座號方向慣例**

- **結果**：測試一寫下去，一壘側內野的 3,420 組相鄰座位（I1-1B）和 1,456 組（I2-1B）全部失敗，每一組都朝右手邊走了 0.5 m；其他四區通過。
- **原因**：照片顯示內野面向場地時座號往左增加，而產生器把一壘側從本壘端往界外桿編號，面向場地時是往右增加。
- **修正**：一壘側改從界外桿那端編起。看台的位置、形狀、席數都不變，資料只有兩區的起訖點對調。
- **怎麼驗證**：紅燈時先確認失敗數量和原因相符；修正後變綠；再用突變確認（內野六區方向全反轉，六條測試全失敗）。測試能抓到錯，才可以相信它說「沒錯」。
- **限制**：這是「記錄我們採用的慣例」，不是驗證官方座號。依據只有我本人坐過的 113 區與相鄰 114 區的照片，外野沒有證據。

**案例 2：敏感度分析（計算「C 級估算」到底會影響結果多少）**

- **方法**：`npm run sensitivity` 把 10 個 C 級參數在範圍內同時、獨立均勻抽樣 1000 次（隨機種子固定為 20261023，結果可重現），每次重建模型並通過 `parseVenue`；被產生器拒絕的樣本 13 次。
- **結果（113 區第 1 排中央，該檔案最後更新的版本）**：距離基準 18.61 m，95% 區間 15.47–19.24 m（±1.88 m）；俯角基準 10.53°，95% 區間 7.71–15.38°（±3.83°）。
- **為什麼是這樣**：距離主要受「B1 前緣距離」影響（擺幅 +3.92 m）；俯角主要受「B1 baseHeight」影響（擺幅 +6.05°），因為俯角由座位相對本壘的高度差決定。「擺幅」的算法是其他參數固定在基準值，只把它從範圍下限改到上限。
- **意義**：估算參數的不確定會放大成約 ±4° 的俯角誤差，所以我對外只稱「近似模型」並附免責聲明。
- **注意**：這份分析的檔案最後更新於 10/24（官方票區版本）；之後 B1 第 1 排降到場地高度等模型改動後需重跑 `npm run sensitivity`，再更新上面的數字。

**案例 3：照片校色**

- **目標**：每個元素、每個通道與參考照片的取樣值差 ≤ 15/255。
- **結果**：Pass 1 在 3 輪後座椅中位群差 +40，原因是渲染的取樣框只框到近處幾張大椅子和踏面；換成框 0.5× 中一片小椅子後再 2 輪全部通過。Pass 2 只剩座椅最亮群差 −22。
- **為什麼還差**：霧把遠處的淺色椅子往牆色拉，而淺色材質已經在亮度上限 215 附近。我決定接受現況，不放寬上限、不改霧，數字與前後截圖在 `docs/look-targets.md`。

## 已知限制

- 大巨蛋排與號是模型推算（C），只保證少數票面標籤存在；視野文章照片說明中的 55 個「區-排-號」，模型放得下 40 個，另 15 個只回報。
- 外野座號方向沒有照片證據；穹頂外殼、螢幕與顏色是依照片目視估計的示意。
- 效能還沒有量化（目視操作沒有卡頓）；目前沒有檢查看台遮蔽。

# 6. Bonus：MCP server

SeatView 的 MCP server 把核心功能包成 5 個唯讀 tool，所有數字都來自 `src/core` 的 `getSeatView` 等函式，跟網頁是同一套算式，所以 AI 助理查到的數字跟畫面上的一致。

| tool | 輸入 | 回傳重點 |
|---|---|---|
| `list_venues` | 無 | 場館、各自的 layout、區數、免責聲明 |
| `list_sections` | `venue`, `layout` | 該 layout 開放的區、樓層、排數、每排席數 |
| `get_seat_view` | `venue`, `layout`, `section`, `row`, `seat` | 座位與相機位置、距離、俯角、水平偏角、方位、免責聲明 |
| `compare_seats` | `venue`, `layout`, `seats`（2–10 筆） | 每席指標、由近到遠與由正到偏的排名 |
| `get_venue_provenance` | `venue` | 每個數字的值、等級（A／B／C）與來源 |

**例子**（真正的 stdio 連線，2026-10-24 的原樣輸出，免責聲明節錄；之後大巨蛋模型有調整，數字以重跑為準）：

```
get_seat_view { venue: "taipei-dome", layout: "baseball", section: "113",
row: 38, seat: 7 }
臺北大巨蛋（近似模型）｜棒球｜113 第 38 排 7 號：距本壘 52.4 m（平面 49.8 m），比本壘高
16.3 m（俯角 18.2°），水平偏角 -0.3°，位於正面。
本模型為依公開報導、公開座位圖的票區編號與棒球規則推算的近似模型……不應作為購票依據。
```

錯誤也設計成 AI 看得懂：例如查演唱會時不開放的 140 區，會回傳 `isError` 與 `SECTION_NOT_IN_LAYOUT`，並列出該 layout 開放的區，讓 agent 可以自己修正再問。

### 設計決策

- **放 `src/mcp`，不放 `packages/mcp`**：直接用 `src/core`，共用 tsconfig、Vitest 與 typecheck。
- **Node 直接跑 TypeScript，不需要 build**：否則改完程式碼忘記重 build 就會跑到舊版。代價是 core 的相對 import 要補 `.ts`（只改 import 字串，函式一個都沒動）。
- **處理函式與 SDK 分開**：`tools.ts` 只用 core、不碰 SDK，方便單元測試；`server.ts` 只負責註冊 tool。
- **venue 用 enum 列出**：AI 不用猜有哪些場館。

- **近似模型一定附免責聲明、提供 `get_venue_provenance`**：避免助理把 C 級估算講成事實。

### 怎麼驗證「agent 可以成功呼叫」

1. **處理函式測試**（`src/mcp/tools.test.ts`）：`get_seat_view` 的數字與 core 的 `getSeatView` 完全一致、方位標籤的分界角度、各種錯誤都附提示、`compare_seats` 中某席出錯不影響其他席。
2. **協定層端到端測試**（`src/mcp/server.test.ts`）：用 SDK 的 `InMemoryTransport` 走完整的 MCP 協定。列出 5 個 tool、都有輸入輸出 schema 與唯讀標註，每個回傳都通過 `outputSchema` 驗證，不合法的輸入在執行 core 之前就被擋下。測試才能只檢查 `isError` 不是 true，是因為我先查了 SDK 原始碼，確認輸入輸出不符 schema 時都會回傳 `isError`。
3. **真正的 stdio**：用 SDK 的 `StdioClientTransport` 啟動 `node src/mcp/main.ts`，244 ms 連上，5 個 tool 各呼叫一次都回傳結構化資料；stdout 只有協定訊息，啟動訊息在 stderr。
4. **大巨蛋的端到端測試**：對大巨蛋資料跑 `get_seat_view`（commit `test(mcp): end-to-end get_seat_view against the Taipei Dome data`）。

**接上 agent 的方式**：

```
claude mcp add seatview -- node C:/seatview/src/mcp/main.ts
```

Claude Desktop 則在 `claude_desktop_config.json` 加入 `{ "mcpServers": { "seatview": { "command": "node", "args": ["C:/seatview/src/mcp/main.ts"] } } }`。需要 Node ≥ 22.18。

**這一版不做**：回傳 render 截圖（需要無畫面的 WebGL 環境）、依條件找座位、HTTP transport、MCP resources 與 prompts。

## 7. 反思與第二階段（棒球版）

棒球版的骨架已經提前完成：大巨蛋已經有 `baseball` layout，可以從座位望向本壘板

| 第二階段項目 | 狀態 | 怎麼擴充 |
| --- | --- | --- |
| 同一場館切換棒球配置、望向本壘板 | 已完成 | 只新增 layout（focus = 本壘）與區的 `layouts` 白名單 |
| 球路軌跡疊加 | 尚未做 | 新增 overlay 型別與 render 端的繪製；core 函式簽名不動（schema 已有 polygon overlay 這類欄位的先例） |

| 第二階段項目 | 狀態 | 怎麼擴充 |
| --- | --- | --- |
| 看台遮蔽、依條件找座位（`find_seats`） | 尚未做 | 在 core 新增函式，再由 MCP 重用 |
| 效能量化 | 尚未做 | 用 DevTools Performance 量 FPS |

## 反思

- **做得對的事**：寫程式前先定死座標慣例與函式簽名；讓網頁和 MCP 共用同一個 `getSeatView`；測試先行加突變確認；把資料分成 A／B／C 等級，讓 C 級估算不會被說成事實。
- **比較不理想的地方**：11/01 大量時間花在視覺擬真（校色、屋頂梁、圍牆通風孔），這些對**座位視野的數字**的幫助比較小；大巨蛋的排、號、高度有不少 C 級估算，準確度受限於公開資料。
- **下次會改**：更早把效能量化；讓敏感度分析在模型改動後自動重跑，不會放著變舊；用截圖自動化取代手動目視清單。
- **對 AI 協作的心得**：AI 很適合把一個清楚的規格做成測試加實作，但 AI 做的哪些部分是對的仍要我來判斷，例如座號方向、要不要接受不對稱的解、校色要不要放寬上限。把每次決策寫成 ai-log，也讓我事後能說明這個專案的哪些部分是我決定的。